

# BHTTP/1 - HTTP, in binary

Protocol specification, version 1. Shreyansh Arora (24bcs10252). MUST, MUST NOT, SHOULD and MAY are as in RFC 2119. Byte-level example: HEXDUMP.md.

## 1. Conventions

BHTTP/1 moves files from a server to a client over one long-lived TCP connection (default port 9000). Integers are unsigned big-endian. Strings are UTF-8, never NUL-terminated, with an explicit length.

## 2. Connection

The client opens one TCP connection and sends the preface 42 48 54 31 (“BHT1”); after it, both sides send only frames. A server that reads any other preface sends GOAWAY(3) and closes. The client MUST NOT open a second connection, with one exception: after GOAWAY(3) in reply to a newer preface (for example “BHT2”) it MAY reconnect once with “BHT1”. This is how version 2 gets in.

Requests run in lockstep: the client MUST NOT send a request until the previous response has ended (END\_STREAM), so at most one stream is open. A server that receives one anyway MAY answer it in order or send GOAWAY(1). The connection stays open after every response. Either side ends it with GOAWAY (section 4). A server MAY close an idle connection with GOAWAY(0), and before reading the preface MAY close without GOAWAY (slow preface, server full).

## 3. Frame header (8 bytes, fixed)

Length (24 bits) | Type (8) | Flags (8) | Stream ID (24), then Length payload bytes. 64 bits: two aligned 32-bit words in a hexdump. Type values are in section 4.

**Length:** payload bytes, excluding the header. Senders MUST NOT exceed 16384, so one file cannot hog the connection and a receiver never buffers more than 8 + 16384 bytes per frame. A receiver checks Length before it looks at Type: above 16384, on any type, known or unknown, it sends GOAWAY(2) and closes without reading the payload. At or below 16384 it MUST read the whole frame. 24 bits lets a later version raise the cap without changing the header.

**Flags:** 0x01 END\_STREAM is the only flag. Senders MUST set the other bits to 0; receivers MUST ignore them.

**Stream ID:** ties a response to its request. The client numbers requests from 1, strictly increasing (gaps allowed); the server answers on the request’s ID. A stream is *idle* until its request is sent, *open* until the response frame carrying END\_STREAM, then *closed*. A skipped lower ID is closed too: the only frame ever sent on a stream that is not open is the 400 that answers it (section 9). ID 0 is for GOAWAY only. When IDs run out (16,777,215) the client sends GOAWAY(0).

**Why these widths.** Type and Flags are one byte each, so every field is byte-aligned; 256 types leave room for version 2, and one flag is used today. Stream ID is 24 bits, not HTTP/2’s 31 plus a reserved bit: in lockstep only one stream is ever open, so the ID catches stale or misdirected frames rather than multiplexing, and 24 bits keep the header at exactly 8 bytes.

## 4. Frame types

**0x0 HEADERS:** one request (client) or one response status and headers (server), in one frame. **0x1 DATA:** response body bytes. Zero or more per response; zero-length is legal. **0x2 GOAWAY:** Length MUST be 2 and Stream ID MUST be 0, else protocol error. Payload: error code 0 normal, 1 protocol error, 2 frame too large, 3 bad preface, 4 internal error; a receiver treats unknown codes as 1. After sending or receiving GOAWAY an endpoint MUST NOT send anything more on the connection; a request still open was not completed and is not retried.

**Unknown types** (Length within the section 3 cap; above it is GOAWAY(2)): a receiver MUST read Length bytes, discard them and carry on, whatever the Stream ID or Flags. It MUST NOT treat this as an error. This is how version 2 adds frame types. A server also skips DATA from a client.

## 5. Request (HEADERS payload, client to server)

Method (1) | Path length (2) | Path | Header count (1) | Header entries

Method: 0x01 GET, 0x02 HEAD. Anything else is malformed. The client MUST set END\_STREAM; a request is always exactly one frame, so the server ignores the flag. Requests have no body. All request headers are optional; a server MUST NOT reject a request for lacking one.

Path: UTF-8, starts with “/”, no NUL, at most 16380 bytes, used literally: no query string, no percent-decoding (“%2e%2e” is a file name). A “..” segment is malformed. The server resolves a path in this order: (1) drop empty and “.” segments (//a/.b is /a/b); (2) “/” and directories map to their index.html; (3) if the path as sent ends in “/” and names a regular file, 404. So /index.html/ is 404 and /index.html/. is the file. A missing or non-regular file, and anything that resolves outside the server root, including through a symlink, are 404.

A request MAY carry the literal header if-none-match; if it equals the file’s etag, or is \*, the answer is 304.

## 6. Response (HEADERS payload, server to client)

Status uses HTTP numbers and is always final: there are no interim 1xx responses. bserve sends 200, 304, 400, 404, 500. A client MUST treat a status outside 100-599 as a protocol error. Every response carries content-length: one to 18 ASCII digits 0-9 (leading zeros allowed); receivers MUST reject any other value (protocol error). A 200 or 304 also carries content-type, last-modified, etag and cache-control; every response carries server and date. Dates are IMF-fixdate (RFC 9110 section 5.6.7).

With a body, HEADERS has no flags, DATA follows, and the last DATA frame carries END\_STREAM; the DATA lengths MUST sum to content-length. Without a body (HEAD, 304, empty file, every error) END\_STREAM is on HEADERS and no DATA follows; for HEAD and 304, content-length is the size GET would return, and errors carry content-length 0. Senders MUST put END\_STREAM on HEADERS when the body is empty; receivers also accept one empty DATA frame carrying END\_STREAM.

## 7. Header entries (HPACK’s first two ideas)

Each entry starts with a 1-byte index. 0x01-0x0A: a static-table name, then Value length (2) and Value. 0x00: a literal name, Name length (1, at least 1), Name, Value length (2), Value. 0x0B-0xFF are reserved: malformed.

|          |              |          |                 |                  |
|----------|--------------|----------|-----------------|------------------|
| 1 host   | 2 user-agent | 3 accept | 4 content-type  | 5 content-length |
| 6 server | 7 date       | 8 etag   | 9 last-modified | 10 cache-control |

bcurl sends 1-3; bserve sends 4-10. Names are ASCII a-z 0-9 - only. A name MUST NOT appear twice (a literal equal to a static name counts as that name); order does not matter; receivers ignore headers they do not use. Values are UTF-8.

**Parsing rule (both sides, every length field):** a length running past the end of the payload (underrun) and bytes left after the last field (overrun) are malformed. Receivers never scan for delimiters. “content-length” is 14 bytes as a literal, 1 byte indexed.

## 8. Example exchanges

- GET /index.html (12 bytes):** request frame header 00 00 30 00 01 00 00 01; response HEADERS (with bserve’s headers) 00 00 79 00 00 00 00 01 then DATA 00 00 0c 01 01 00 00 01 + 12 bytes.
- GET of 40,000 bytes:** HEADERS (flags 0), DATA of 16384, 16384 and 7232 bytes, END\_STREAM on the last. Boundary: a 16384-byte file is exactly one DATA frame.
- HEAD, 304 or empty file:** one HEADERS frame with END\_STREAM and no DATA.
- Malformed, then valid:** stream 1 with method 0x07 gets 400 on stream 1; stream 2 GET / then gets 200. The connection survives a 400.

## 9. Errors

**Received by a server.** *Bad payload, framing intact* (bad method or path, underrun, overrun, reserved index, invalid UTF-8, bad or duplicate name, a request ID not above every earlier one, including a repeat): 400 on that ID, connection stays open. *Missing file:* 404. *File fails before the response starts:* 500. *File fails after it starts:* GOAWAY(4). *Bad preface:* GOAWAY(3). *Length above 16384:* GOAWAY(2). *HEADERS on stream 0, malformed GOAWAY, connection cut inside a frame, or a frame not completed in time:* GOAWAY(1).

**Received by a client.** GOAWAY(1), then close, for: DATA before HEADERS; a second HEADERS; a HEADERS or DATA frame for any stream other than the one the client is waiting on; a malformed response payload; a status outside 100-599; a missing content-length, or one that is not 1 to 18 ASCII digits; on a HEAD or 304 response, HEADERS without END\_STREAM or any DATA; DATA totalling more or less than content-length; a frame not completed in time. A received GOAWAY ends the client without a reply. After its last response a client MAY send GOAWAY(0) without reading further.

## 10. Limits and fault behavior

- No connection** (unreachable, name not found): nothing is sent; bcurl exits 1. Trying the next address of one name is not a second connection.
- Timeouts:** a receiver MAY give up on a frame not completed in time (bserve: 10 s from its first byte, then GOAWAY(1)). bserve closes a connection with no new request for 30 s after the last response ended with GOAWAY(0). A client SHOULD give up after a timeout of its choosing (bcurl: 30 s without receiving a frame) and send GOAWAY(0).
- Over-length response:** the client stops before writing bytes beyond content-length.
- Reset or close without GOAWAY:** the open request failed and is not retried.
- Graceful shutdown:** the GOAWAY sender half-closes (FIN) right after it, then discards input until the peer closes (bserve and bcurl: at most 1 s or 64 KiB).